

HOMWORK 4: LOGISTIC REGRESSION *

10-301 / 10-601 INTRODUCTION TO MACHINE LEARNING (SPRING 2025)

<http://www.cs.cmu.edu/~mgormley/courses/10601/>

OUT: Monday, February 17

DUE: Wednesday, February 26 at 11:59 PM

TAs: Shivi, Albert, Vianna, Andra, Andrew, Markov, Neural

Summary In this assignment, you will build a sentiment polarity analyzer, which will be capable of analyzing the overall sentiment polarity (positive or negative) for restaurant reviews using logistic regression. In the written component, you will study linear and logistic regression.

START HERE: Instructions

- **Collaboration Policy:** Please read the collaboration policy here: <http://www.cs.cmu.edu/~mgormley/courses/10601/syllabus.html>
- **Late Submission Policy:** See the late submission policy here: <http://www.cs.cmu.edu/~mgormley/courses/10601/syllabus.html>
- **Submitting your work:** You will use Gradescope to submit answers to all questions and code. Please follow instructions at the end of this PDF to correctly submit all your code to Gradescope.
 - **Written:** For written problems such as short answer, multiple choice, derivations, proofs, or plots, please use the provided template. Submissions can be handwritten onto the template, but should be labeled and clearly legible. If your writing is not legible, you will not be awarded marks. Alternatively, submissions can be written in $\text{\LaTeX}$. Each derivation/proof should be completed in the boxes provided. You are responsible for ensuring that your submission contains exactly the same number of pages and the same alignment as our PDF template. If you do not follow the template, your assignment may not be graded correctly by our AI assisted grader and there will be a **2% penalty** (e.g., if the homework is out of 100 points, 2 points will be deducted from your final score).
 - **Programming:** You will submit your code for programming questions on the homework to [Gradescope](#). After uploading your code, our grading scripts will autograde your assignment by running your program on a virtual machine (VM). You are only permitted to use [the Python Standard Library modules](#) and `numpy`.

Ensure that the version number of your programming language environment (i.e. Python 3.9.12) and versions of permitted libraries (i.e. `numpy` 1.23.0) match those used on Gradescope. You

*Compiled on Thursday 27th February, 2025 at 02:24

have 10 free Gradescope programming submissions, after which you will begin to lose points from your total programming score. We recommend debugging your implementation on your local machine (or the Linux servers) and making sure your code is running correctly first before submitting your code to Gradescope.

- **Materials:** The data and reference output that you will need in order to complete this assignment is posted along with the writeup and template on the course website.

Instructions for Specific Problem Types

For “Select One” questions, please fill in the appropriate bubble completely:

Select One: Who taught this course?

- ☒ Matt Gormley
- ☐ Marie Curie
- ☐ Noam Chomsky

If you need to change your answer, you may cross out the previous answer and bubble in the new answer:

Select One: Who taught this course?

- ☒ Henry Chai
- ☐ Marie Curie
- ☒ Noam Chomsky

For “Select all that apply” questions, please fill in all appropriate squares completely:

Select all that apply: Which are instructors for this course?

- ☒ Matt Gormley
- ☒ Henry Chai
- ☐ Noam Chomsky
- ☐ I don't know

Again, if you need to change your answer, you may cross out the previous answer(s) and bubble in the new answer(s):

Select all that apply: Which are the instructors for this course?

- ☒ Matt Gormley
- ☒ Henry Chai
- ☒ Noam Chomsky
- ☒ I don't know

For questions where you must fill in a blank, please make sure your final answer is fully included in the given space. You may cross out answers or parts of answers, but the final answer must still be within the given space.

Fill in the blank: What is the course number?

10-601

10-~~6~~301

Written Questions (42 points)

1 $\text{\LaTeX}$ Point and Template Alignment (1 points)

1. (1 point) **Select one:** Did you use $\text{\LaTeX}$ for the entire written portion of this homework?

☒ Yes

☐ No

2. (0 points) **Select one:** I have ensured that my final submission is aligned with the original template given to me in the handout file and that I haven't deleted or resized any items or made any other modifications which will result in a misaligned template. I understand that incorrectly responding yes to this question will result in a penalty equivalent to 2% of the points on this assignment.

Note: Failing to answer this question will not exempt you from the 2% misalignment penalty.

☒ Yes

3. (0 points) **Select one:** Did you fill out the [Exit Poll](#) for the previous HW? Completing the exit poll will count towards your participation grade.

☒ Yes

2 Linear Regression (4 points)

1. We would like to fit a linear regression model to the dataset

$$\mathcal{D} = \left\{ \left(\mathbf{x}^{(1)}, y^{(1)} \right), \left(\mathbf{x}^{(2)}, y^{(2)} \right), \dots, \left(\mathbf{x}^{(N)}, y^{(N)} \right) \right\}$$

with $\mathbf{x}^{(i)} \in \mathbb{R}^M$ by minimizing the ordinary least square (OLS) objective function:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - \sum_{j=1}^M w_j x_j^{(i)} \right)^2.$$

- (a) (2 points) **Select one:** We solve for each coefficient w_k ($1 \leq k \leq M$) by deriving an expression of w_k from the critical point $\frac{\partial J(\mathbf{w})}{\partial w_k} = 0$. What is the expression for each w_k in terms of the dataset $(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})$ and $w_1, \dots, w_{k-1}, w_{k+1}, \dots, w_M$?

- ☒ $w_k = \frac{\sum_{i=1}^N x_k^{(i)} (y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)})}{\sum_{i=1}^N (x_k^{(i)})^2}$
- ☐ $w_k = \frac{\sum_{i=1}^N x_k^{(i)} (y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)})}{\sum_{i=1}^N (y^{(i)})^2}$
- ☐ $w_k = \frac{\sum_{i=1}^N x_k^{(i)} (y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)})}{\sum_{i=1}^N (x_k^{(i)} y^{(i)})^2}$
- ☐ $w_k = \sum_{i=1}^N x_k^{(i)} (y^{(i)} - \sum_{j=1}^M w_j x_j^{(i)})$

- (b) (1 point) **Select one:** How many coefficients (w_k) do you need to estimate? When solving for these coefficients, how many equations do you have?

- ☒ M coefficients, M equations
- ☐ M coefficients, N equations
- ☐ N coefficients, M equations
- ☐ N coefficients, N equations

2. (1 point) **Select all that apply:** The convexity of functions plays an important role when optimizing machine learning models. Given the statements about convexity below, select all that are true:

- ☐ A strictly convex function can have infinite minimizers, while a convex function can have only zero or one minimizer.
- ☒ The Mean Absolute Error (MAE) loss function, defined as $L(y, \hat{y}) = \sum_{i=1}^N |y_i - \hat{y}_i|$, is a non-convex function.
- ☒ Convex loss functions avoid the problem of multiple different local minima, making it easier to converge on the optimal solution.
- ☐ The L2 regularization term, $\lambda \|\theta\|_2^2$, is convex but not strictly convex.

3 Logistic Regression: Warm-Up (5 points)

The following questions should be completed before you start the programming component of this assignment.

The following dataset consists of 4 training examples, where $x_k^{(i)}$ denotes the k -th dimension of the i -th training example $\mathbf{x}^{(i)}$, and $y^{(i)}$ is the corresponding label ($k \in \{1, 2, 3\}$ and $i \in \{1, 2, 3, 4\}$).

i	x_1	x_2	x_3	y
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0

A binary logistic regression model is trained on this dataset, and the parameter vector θ after training is

$$\theta = [1.5 \quad 2 \quad 1]^T.$$

Note: There is **no intercept term** used in this problem.

Use the data above to answer the following questions. For all numerical answers, please use one number rounded to the fourth decimal place; e.g., 0.1234. Showing your work in these questions is optional, but it is recommended to help us understand where any misconceptions may occur.

- (2 points) Calculate $J(\theta)$, $\frac{1}{N}$ times the negative log-likelihood over the given data and parameter θ . (Note here we are using natural log, i.e., the base is e).

$J(\theta)$
0.7969

Work

2. (2 points) Calculate the gradients $\frac{\partial J(\theta)}{\partial \theta_j}$ with respect to θ_j for all $j \in \{1, 2, 3\}$.

$\partial J(\theta)/\partial \theta_1$	$\partial J(\theta)/\partial \theta_2$	$\partial J(\theta)/\partial \theta_3$
0.2044	0.04165	0.1709

Work

3. (1 point) Update the parameters following the parameter update step $\theta_j \leftarrow \theta_j - \eta \frac{\partial J(\theta)}{\partial \theta_j}$ and write the updated (numerical) value of the vector θ . Use learning rate $\eta = 1$.

θ_1	θ_2	θ_3
1.2956	2.04165	0.8291

Work

4 Logistic Regression: Analysis (7 points)

1. (2 points) **Select all that apply:** Which of the following are true about logistic regression?

- ☒ Our formulation of binary logistic regression will work with both continuous and binary features.
- ☒ Binary Logistic Regression will form a linear decision boundary in our feature space, assuming no feature engineering.
- ☒ The sigmoid function is convex.
- ☐ The negative log-likelihood function for logistic regression is not convex so gradient descent may get stuck in a sub-optimal local minimum.
- ☐ None of the above.

2. (1 point) **Select one:** The *average* negative log-likelihood $J(\boldsymbol{\theta})$ for binary logistic regression can be expressed as

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \left[-y^{(i)} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} \right) + \log \left(1 + \exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) \right) \right]$$

where $\mathbf{x}^{(i)} \in \mathbb{R}^{M+1}$ is the column vector of the feature values of the i -th data point, $y^{(i)} \in \{0, 1\}$ is the i -th class label, $\boldsymbol{\theta} \in \mathbb{R}^{M+1}$ is the weight vector. When we want to perform logistic ridge regression (i.e. with ℓ_2 regularization), we modify our objective function to be

$$f(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \frac{1}{2} \sum_{j=0}^M \theta_j^2$$

where λ is the regularization weight, θ_j is the j th element in the weight vector $\boldsymbol{\theta}$. Suppose we are updating θ_k with learning rate η , which of the following is the correct expression for the update?

- ☐ $\theta_k \leftarrow \theta_k + \eta \frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k}$ where $\frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(y^{(i)} - \frac{\exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})}{1 + \exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})} \right) \right] + \lambda \theta_k$
- ☐ $\theta_k \leftarrow \theta_k + \eta \frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k}$ where $\frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(-y^{(i)} + \frac{\exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})}{1 + \exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})} \right) \right] - \lambda \theta_k$
- ☐ $\theta_k \leftarrow \theta_k - \eta \frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k}$ where $\frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(-y^{(i)} + \frac{\exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})}{1 + \exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})} \right) \right] + \lambda \theta_k$
- ☒ $\theta_k \leftarrow \theta_k - \eta \frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k}$ where $\frac{\partial f(\boldsymbol{\theta})}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(-y^{(i)} - \frac{\exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})}{1 + \exp(\boldsymbol{\theta}^T \mathbf{x}^{(i)})} \right) \right] + \lambda \theta_k$

3. (2 points) Data is separable in one dimension if there exists a threshold t such that all values less than t have one class label and all values greater than or equal to t have the other class label. If you train an unregularized logistic regression model for infinite iterations on training data that is separable in at least one dimension, the corresponding weight(s) can go to infinity in magnitude. What is an explanation for this phenomenon?

Hint: Suppose you find a weight vector yielding a decision boundary that fully separates a dataset. Think about what the probability for each point is and whether you could adjust the weights to make the overall likelihood more favorable.

Your Answer

I believe this question is basically describing overfitting. When the model increase the magnitude of the weights (to infinity) which then creates a complex model but fits all data points (even noisy ones) to perfectly fit all data points. This causes weights to grow infinitely.

4. (2 points) **Select all that apply:** How does regularization (such as ℓ_1 and ℓ_2) help correct the problem in the previous question?
- ☐ ℓ_1 regularization prevents weights from going to infinity by penalizing the count of non-zero weights.
 - ☒ ℓ_1 regularization prevents weights from going to infinity by reducing some of the weights to 0, effectively removing some of the features.
 - ☒ ℓ_2 regularization prevents weights from going to infinity by reducing the value of some of the weights to *close* to 0 (reducing the effect of a feature but not necessarily removing it).
 - ☐ None of the above.

5 Logistic Regression: Adversarial Attack (6 points)

A grayscale image can be represented numerically as a matrix of intensity values. Each element in the matrix represents a pixel with intensity value in the continuous range $[0, 1]$, zero being the darkest. We can flatten that matrix into a vector. Image classification tasks then use this vector of pixel values as features to predict an image label. For example, consider a small picture that consists of 4 pixels with the corresponding intensity values

0.0	0.1
0.5	1.0

In order to input this into our model, we could create the vector $\mathbf{x} = [0.0 \ 0.5 \ 0.1 \ 1.0]$ where each element in the vector corresponds to a specific pixel in the image, arranged in column-major order.

Consider a logistic regression model whose purpose is to identify narwhals in grayscale images. The model outputs $\mathbf{y}^{(i)} = 1$ when it predicts the input image contains a narwhal. So we can represent the probability function for whether logistic regression predicts a narwhal as

$$p(y = 1 | \mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$$

where $\boldsymbol{\theta}$ is the vector of learned coefficients and $\mathbf{x}$ is a vector of pixels representing the input image.

- After training the model on a training dataset, we arrive at a set of parameters $\boldsymbol{\theta}$. Given the model parameters $\boldsymbol{\theta}$ and the probability function as defined above, we wish to find an input vector $\mathbf{x}$ that causes the model to make a false prediction of a narwhal. We will do this by using gradient ascent *on our input* $\mathbf{x}$, keeping $\boldsymbol{\theta}$ fixed, to maximize the probability that $\mathbf{x}$ is assigned to the narwhal class.

- (1 point) **Select all that apply:** Gradient *ascent* can be used to find a local maximum; and gradient *descent* to find a local minimum. Which of the following optimization problems would return the $\mathbf{x}$ that maximizes the probability of the narwhal class?

- ☒ $\operatorname{argmax}_{\mathbf{x}} p(y = 1 | \mathbf{x}, \boldsymbol{\theta})$
- ☐ $\operatorname{argmax}_{\mathbf{x}} -p(y = 1 | \mathbf{x}, \boldsymbol{\theta})$
- ☐ $\operatorname{argmin}_{\mathbf{x}} p(y = 1 | \mathbf{x}, \boldsymbol{\theta})$
- ☐ $\operatorname{argmin}_{\mathbf{x}} -p(y = 1 | \mathbf{x}, \boldsymbol{\theta})$
- ☐ None of the above.

- (2 points) **Select all that apply:** Given the setup, write (1) the gradient of the probability function with respect to $\mathbf{x}$ and (2) the *gradient ascent* update rule for $\mathbf{x}$. Define the learning rate to be η .

Gradient of the probability function with respect to x

$$p(y = 1 | x; \theta) = \sigma(\theta^T x)(1 - \sigma(\theta^T x)) \cdot \theta$$

Update Rule

2. (1 point) Using the parameter values θ , directly define a vector $\mathbf{x}$ that maximizes the probability of the narwhal class. You should not use any gradient calculations, nor should you use any iterative updates to compute θ . Assume that none of the elements of θ are 0. Remember that we require the feature values to be in the range $[0, 1]$ to generate an image. Give your answer by expressing each element x_j in terms of the parameter vector θ , indicator function $\mathbb{I}$ and any constants you may need.

Hint: $\mathbb{I}[k > l]$ is an indicator function such that if $k > l$ then it returns value 1 and 0 otherwise. Replace $k > l$ with an appropriate condition.

Your Answer

3. (2 points) **Select one:** Now let's consider whether logistic regression is well-suited for this task. Suppose photos of the exact same white narwhal in a dark ocean background was used to generate the training set. The training photos were captured with the side view of the narwhal centered in the photo at a distance of between 30-50 meters from the camera. Which of the below descriptions of a **test image**, if any, would the model be most likely to predict as "narwhal"?

- ☐ A new photo with the same narwhal in the upper right corner of the image.
- ☐ Identical to one of the training photos, but the narwhal replaced with an equal size white cardboard cutout of the narwhal.
- ☒ Identical to one of the training photos, but the background changed to white.
- ☐ None of the above.

6 Vectorization and Pseudocode (6 points)

The following questions should be completed before you start the programming component of this assignment. Assume the dtypes of all ndarrays are `np.float64`. Vectors are 1D ndarrays.

- (2 points) **Select all that apply:** Consider a matrix $\mathbf{X} \in \mathbb{R}^{N \times M}$ and vector $\mathbf{v} \in \mathbb{R}^M$. We can create a new vector $\mathbf{u} \in \mathbb{R}^N$ whose i -th element is the dot product between $\mathbf{v}$ and the i -th row of $\mathbf{X}$ using NumPy as follows:

```
# X and v are numpy ndarrays
# X.shape == (N, M), v.shape == (M,)
u = np.zeros(X.shape[0])
for i in range(X.shape[0]):
    for j in range(X.shape[1]):
        u[i] += X[i, j] * v[j]
```

Which of the following produce the same result?

- ☒ `u = X @ v`
- ☐ `u = v @ X`
- ☒ `u = np.matmul(X, v)`
- ☐ `u = np.matmul(v, X)`
- ☐ `u = X * v`
- ☐ `u = v * X`
- ☒ `u = np.dot(X, v)`
- ☐ `u = np.dot(v, X)`
- ☐ None of the above.

- Consider a matrix $\mathbf{X} \in \mathbb{R}^{N \times M}$ and vector $\mathbf{w} \in \mathbb{R}^N$. Let $\mathbf{\Omega} = \sum_{i=0}^{N-1} w_i (\mathbf{x}_i - \bar{\mathbf{x}}_i) (\mathbf{x}_i - \bar{\mathbf{x}}_i)^T$ where $\mathbf{x}_i \in \mathbb{R}^M$ is the *column* vector denoting the i -th row of $\mathbf{X}$, $\bar{\mathbf{x}}_i \in \mathbb{R}$ is the mean of $\mathbf{x}_i$, and $w_i \in \mathbb{R}$ is the i -th element of $\mathbf{w}$ ($i \in \{0, 1, \dots, N-1\}$). For the following questions, use `X` and `w` for $\mathbf{X}$ and $\mathbf{w}$, respectively. `X.shape == (N, M)`, `w.shape == (N,)`.

- (2 points) Select the line(s) of valid Python code that constructs a matrix whose i -th row is $(\mathbf{x}_i - \bar{\mathbf{x}}_i)^T$.

- ☐ `(X - np.mean(X, axis=0)).T`
- ☒ `X - np.mean(X, axis=1, keepdims=True)`
- ☐ `X - np.mean(X, axis=0, keepdims=True)`
- ☒ `X - np.expand_dims(np.mean(X, axis=1), 1)`
- ☐ None of the above.

- (2 points) Assume the results from (a) is stored in `M`. Select the line(s) of valid Python code that computes $\mathbf{\Omega}$ from `M`.

- ☐ `np.matmul(w * M.T, M)`
- ☒ `np.matmul(w * M, M.T)`
- ☒ `np.dot(w * M, M.T)`
- ☐ `w * np.dot(M.T, M)`
- ☐ None of the above.

7 Word Embeddings and Gender Biases (4 points)

Word embeddings, such as GloVe embeddings, have important applications in Natural Language Processing because of their ability to capture relationships between words (as you will see in the programming component of this assignment). However, research shows that social biases, such as gender-based stereotypes, can be reflected in these embeddings. Here, you will be exploring word analogies with a word analogy visualizer (<https://lamiyowce.github.io/word2viz/>).

- (2 points) Using the "Add pair" button, add the phrases "Programmer" and "Homemaker" to the diagram. Describe the relative location of those words as it relates to the other words in the diagram. Discuss why this relationship might be problematic **in the context of an ML model**. Optionally, you can refer to this paper (where this particular pair of phrases comes from) for some additional insights: <https://arxiv.org/pdf/1607.06520.pdf>

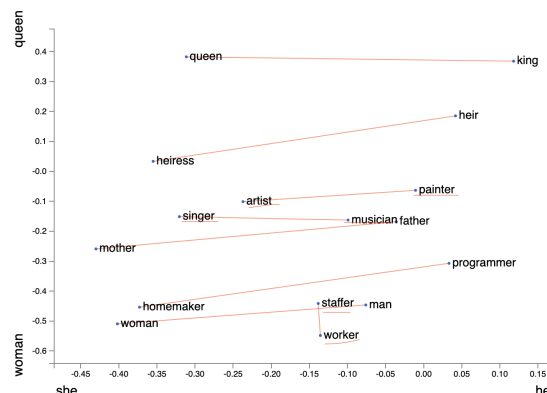
Your Answer

"Homemaker" is positioned in the bottom left corner insinuating that is highly a female related position. "Programmer" is towards the right most side of the graph insinuating that is a male related position. This is problematic since it is biased and categorizes these positions as either one or the other in terms of gender. However "programmers" can be both women and men and the same can be true of "homemakers".

- (2 points) Now, experiment with some additional word or phrase pairs and see where on the graph they are placed based on their embeddings. Do you see any words that you expect to be neutral to be more closely related/biased toward a particular gender? Attach a screenshot of the plot with at least three additional pairs added to it.

Your Answer

Pairs:(musician, singer),(artist, painter),(staffer, worker).



8 Programming Empirical Questions (9 points)

The following questions should be completed as you work through the programming component of this assignment. **Please ensure that all plots are computer-generated.** For all the questions below, unless otherwise specified, use the constant learning rate 0.1.

1. (2 points) 'Using the data in the `largedata` folder in the handout, make a plot that shows the *average* negative log-likelihood for the training and validation data sets after each of 1,000 epochs. The *y*-axis should show the negative log-likelihood and the *x*-axis should show the number of epochs. Your graph should have appropriate scaling and needs to clearly display relevant trends.

Your Answer



2. (2 points) Write a few sentences explaining the output of the above experiment. In particular, do the training and validation log-likelihood curves look the same, or different? Why?

Your Answer

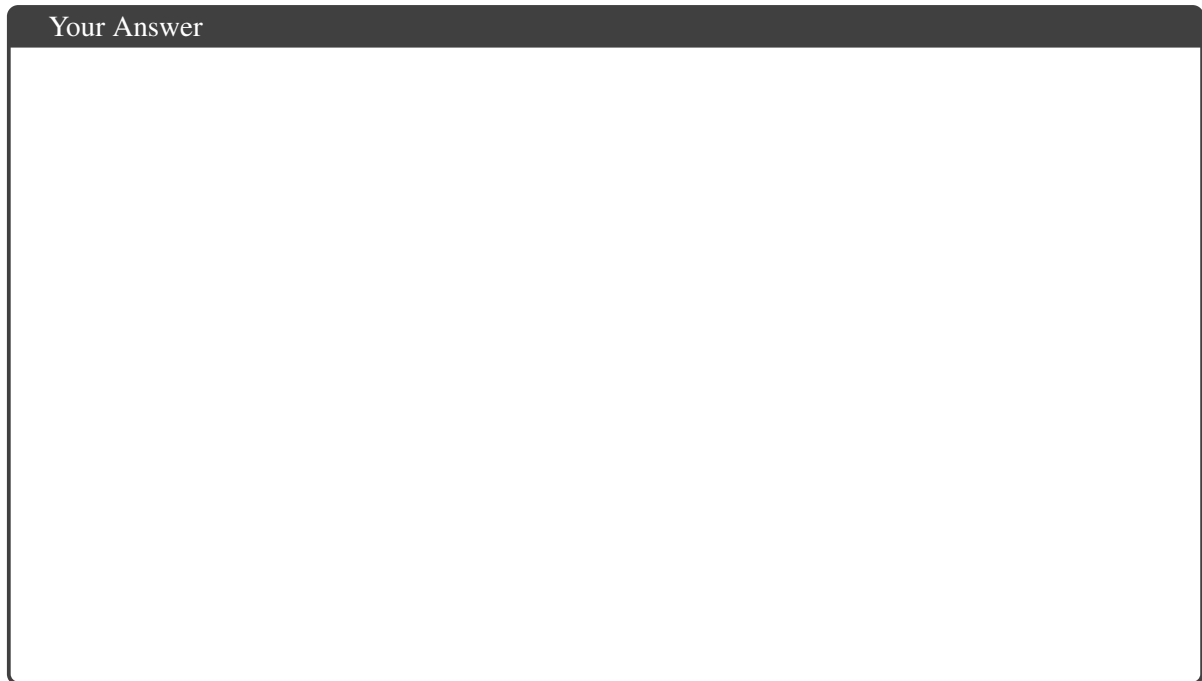


3. (2 points) Report your train and test error for the large data set (found in the `largedata` folder in the handout) after running for 1,000 epochs. Please round to the fourth decimal place, e.g., 0.1234.

Train Error	Test Error
0.0	0.1375

4. (2 points) Using the data in the `largedata` folder of the handout, make a plot comparing the *training* average negative log-likelihood over epochs for three different values for the learning rates, $\eta \in \{10^{-1}, 10^{-2}, 10^{-3}\}$. The y -axis should show the *average* negative log-likelihood, the x -axis should show the number of epochs (from 0 to 1,000 epochs), and the plot should contain three curves corresponding to the three values of η . Provide a legend that indicates the learning rate η for each curve. Your graph should have appropriate scaling and needs to clearly display relevant trends.

Your Answer



5. (1 point) Compare how quickly each curve in the previous question converges.

Your Answer



9 Collaboration Questions

After you have completed all other components of this assignment, report your answers to these questions regarding the collaboration policy. Details of the policy can be found [here](#).

1. Did you receive any help whatsoever from anyone in solving this assignment? If so, include full details.
2. Did you give any help whatsoever to anyone in solving this assignment? If so, include full details.
3. Did you find or come across code that implements any part of this assignment? If so, include full details.

Your Answer

1. No
2. No
3. Any psuedocode that helped explain processes or calculations from the recitations. I also used some of the psuedocode or lines of code that would result in the same output given in part 6 of this homework.

10 Programming (75 points)

Your goal in this assignment is to implement a working Natural Language Processing (NLP) system using binary logistic regression. Your algorithm will determine whether a restaurant review is positive or negative.

Note: Before starting the programming, you should work through the written component to get a good understanding of important concepts that are useful for this programming component.

10.1 The Task

Datasets Download the zip file from the course website, which contains the data for this assignment. This data comes from the Yelp dataset.¹ In the data files, each line is a single example that consists of a label (0 for negative reviews and 1 for positive ones) and a set of words. The format of each example (each line) is `label\tword1 word2 word3 ... wordN\n`, where words are separated from each other with white-space and the label is separated from the words with a tab character.

Examples of the data are as follows:

```
1    i will never forget this single breakfast experience in mad...
0    the search for decent chinese takeout in madison continues ...
0    sorry but me julio fell way below the standard even for med...
1    so this is the kind of food that will kill you so there s t...
```

Feature Engineering In lecture, we saw that we can apply logistic regression to real-valued inputs of fixed length (e.g. $\mathbf{x}^{(i)} \in \mathbb{R}^n$). However, each review has variable length and is not real-valued.

To be able to run logistic regression on the dataset, we first need to transform it using some basic feature engineering techniques. In this homework, we will use a word embeddings model, described in full detail in the next section (10.2).

Programs At a high level, you will write two programs for this homework: `feature.py` and `lr.py`. `feature.py` takes in the raw input data and produces a real-valued vector for each training, validation, and test example. `lr.py` then takes in these vectors and trains a logistic regression model to predict whether each example is a positive or negative review.

10.2 Feature Model

In order to transform a set of words into vectors, we rely on a popular method of feature engineering: word embeddings.

We use ϕ to denote a feature engineering method and $\mathbf{x}^{(i)}$ to denote a training example (a set of English words as seen in 10.1).

Rather than simply indicating which words are present, word embeddings represent each word by “embedding” it into a low-dimensional vector space, which may carry more information about the semantic meaning of the word. In this homework, we use the *GloVe* embeddings, a commonly used set of feature vectors.²

Embeddings `glove-embeddings.txt` contains the *GloVe* embeddings of 6792 words. Not every word in each review is present in the provided `glove-embeddings.txt` file. We treat such missing

¹For more details, see <https://www.yelp.com/dataset>.

²For more details on how these embeddings were trained, see the original work at <https://nlp.stanford.edu/projects/glove/>

words as “out-of-vocabulary” and ignore them. Each line consists of a word and its embedding separated by tabs:

`word\tfeature1\tfeature2\t...\tfeature300\n`. Each word’s embedding is always a 300-dimensional vector. As an example, here are the first few lines of `glove_embeddings.txt`, with values rounded to 3 decimal places:

deserves	0.175	-0.153	-0.208	0.092	0.222	0.202	...
butter	0.357	0.469	-0.021	0.024	-0.168	-0.213	...
staffing	-0.076	0.212	-0.384	0.552	-0.193	-0.052	...
weird	0.110	0.090	0.139	0.340	-0.098	-0.113	...

Using Word Embeddings For this model, there will be two steps in the feature engineering process:

1. First, we would like to exclude words from the review that are not included in the *GloVe* dictionary. Let $\mathbf{x_trim}^{(i)} = \text{TRIM}(\mathbf{x}^{(i)})$, where $\text{TRIM}(\mathbf{x}^{(i)})$ trims the list of words $\mathbf{x}^{(i)}$ by only including words of $\mathbf{x}^{(i)}$ present in `glove_embeddings.txt`.
2. Second, we want to take the trimmed vector $\mathbf{x_trim}^{(i)}$ and convert it to the final feature vector by averaging the *GloVe* embeddings of its words:

$$\phi(\mathbf{x}^{(i)}) = \frac{1}{J} \sum_{j=1}^J \text{GloVe}(\mathbf{x_trim}_j^{(i)})$$

where J denotes the number of words in $\mathbf{x_trim}^{(i)}$ and $\mathbf{x_trim}_j^{(i)}$ is the j -th word in $\mathbf{x_trim}^{(i)}$.

In the given equation, $\text{GloVe}(\mathbf{x_trim}_j^{(i)}) \in \mathbb{R}^{300}$ is the *GloVe* feature vector for the word $\mathbf{x_trim}_j^{(i)}$.

The following **example** provides a reference:

- Let $\mathbf{x}^{(i)}$ denote the sentence “a hot dog is not a sandwich because it is not square”.
- A toy *GloVe* dictionary is given as follows:

hot	0.1	0.2	0.3
not	-0.1	0.2	-0.3
sandwich	0.0	-0.2	0.4
square	0.2	-0.1	0.5

- Then, $\mathbf{x_trim}^{(i)}$ denotes the trimmed review “hot not sandwich not square”. In this trimmed text, the words that are not in the *GloVe* dictionary are excluded. Also note that we keep the order of words and do not de-duplicate words in the trimmed text.³
- The feature for $\mathbf{x}^{(i)}$ can be calculated as

$$\begin{aligned} \phi_2(\mathbf{x}^{(i)}) &= \frac{1}{5} (\text{GloVe}(\text{hot}) + 2 \cdot \text{GloVe}(\text{not}) + \text{GloVe}(\text{sandwich}) + \text{GloVe}(\text{square})) \\ &= [0.02 \quad 0.06 \quad 0.12]^T. \end{aligned}$$

³Keeping duplicates is equivalent to weighting words by their frequency. If “good” appears 3 times as often as “bad”, the movie review is more likely to be positive than negative.

10.3 `feature.py`

`feature.py` implements word embeddings (described above in 10.2) to transform raw training examples (a label and a list of English words) to formatted training examples (a label and a feature vector).

Inputs

- **Input data** for training, validation, and testing. Each data point contains a label and an English restaurant review in the format described in 10.1.
- **GloVe embeddings** to use for the word embedding feature extraction methods.

Outputs

- **Formatted data** for training, validation, and testing. You should perform feature extraction on *each* of the training, validation, and test sets.

Output Format Each output file (one for training data, one for validation, and one for testing) should contain the formatted presentation of each example printed on a new line. Use `\n` to create a new line. The format for each line should exactly match `label\tvalue1\tvalue2\tvalue3\t...\valueM\n`.

Each line corresponds to a particular restaurant review, where the first entry is the label and the rest are the features in the feature vector. The rows are the summed up *GloVe* vectors for all the words present in the dictionary. All entries are separated with a tab character. The handout folder contains example formatted outputs on the small dataset; they are partially reproduced below for your reference. Please round your outputs to 6 decimal places.

1.000000	-0.166646	0.641027	-0.064805	...
0.000000	-0.224874	0.461526	-0.215232	...
0.000000	-0.222178	0.437475	-0.083073	...
1.000000	-0.215923	0.612535	0.061671	...

10.4 `lr.py`

`lr.py` implements a logistic regression classifier that takes in formatted training data and produces a label (either 0 or 1) that corresponds to whether each restaurant review was negative or positive. **Inputs**

- **Formatted data** for training, validation, and testing. Each data point contains a label and a corresponding feature vector. These files are the ones produced by `feature.py`.
- **The number of epochs** to train for, which will be passed in as a command line argument.
- **The learning rate**, also passed in via the command line.

Requirements

- Include an intercept term in your model. You can either treat the intercept term as a separate variable, or fold it into the parameter vector (recommended). In either case, make sure you update the intercept parameter correctly.
- Initialize all model parameters to 0.
- Use stochastic gradient descent (SGD) to train the logistic regression model.

- Perform SGD updates on the training data **in the order that the data is given in the input file**. While we would normally shuffle training examples in SGD, we need training to be deterministic in order to autograde this assignment. **Do not shuffle the training data.**

Outputs

- **Labels** for the training and testing data.
- **Metrics** for the training and testing error.

Output Labels Format Your `lr` program should produce two output `.txt` files containing the predictions of your model on training data and test data. Each file should contain the predicted labels for each example printed on a new line. The name of these files will be passed as command line arguments. Use `\n` to create a new line. An example of the labels is given below.

```
1
0
0
1
```

Output Metrics Format Your program should generate a `.txt` file where you report the final training and testing error after training has completed. The name of this file will be passed as a command line argument.

All of your reported numbers should be within 0.00001 of the reference solution, and you should round the error values to 6 decimal places. The following example is the reference solution for the small dataset after 500 training epochs with learning rate 0.1.

```
error(train): 0.000000
error(test): 0.500000
```

Each line in the output file should be terminated by a newline character `\n`. There is a whitespace character after the colon.



Figure 1: Programming pipeline for sentiment analyzer based on binary logistic regression

10.5 Command Line Arguments

The autograder runs and evaluates the output from the files generated, using the following command (note feature will be run before lr):

```
$ python feature.py [args1...]  
$ python lr.py [args2...]
```

Where above [args1...] is a placeholder for seven command-line arguments: <train_input> <validation_input> <test_input> <feature_dictionary_input> <formatted_train_out> <formatted_validation_out> <formatted_test_out> . These arguments are described in detail below:

1. <train_input>: path to the training input .tsv file (see Section 10.1)
2. <validation_input>: path to the validation input .tsv file (see Section 10.1)
3. <test_input>: path to the test input .tsv file (see Section 10.1)
4. <feature_dictionary_input>: path to the *GloVe* feature dictionary .txt file (see Section 10.2)
5. <formatted_train_out>: path to output .tsv file to which the feature extractions on the *train-*ing data should be written (see Section 10.3)

6. `<formatted_validation_out>`: path to output `.tsv` file to which the feature extractions on the *validation* data should be written (see Section 10.3)
7. `<formatted_test_out>`: path to output `.tsv` file to which the feature extractions on the *test* data should be written (see Section 10.3)

Likewise, `[args2...]` is a placeholder for eight command-line arguments: `<formatted_train_input>` `<formatted_validation_input>` `<formatted_test_input>` `<train_out>` `<test_out>` `<metrics_out>` `<num_epoch>` `<learning_rate>`. These arguments are described in detail below:

1. `<formatted_train_input>`: path to the formatted training input `.tsv` file (see Section 10.3)
2. `<formatted_validation_input>`: path to the formatted validation input `.tsv` file (see Section 10.3)
3. `<formatted_test_input>`: path to the formatted test input `.tsv` file (see Section 10.3)
4. `<train_out>`: path to output `.txt` file to which the prediction on the *training* data should be written (see Section 10.4)
5. `<test_out>`: path to output `.txt` file to which the prediction on the *test* data should be written (see Section 10.4)
6. `<metrics_out>`: path of the output `.txt` file to which metrics such as train and test error should be written (see Section 10.4)
7. `<num_epoch>`: integer specifying the number of times SGD loops through all of the training data (e.g., if `<num_epoch>` equals 5, then each training example will be used in SGD 5 times).
8. `<learning_rate>`: float specifying the learning rate; in the reference output, we set the learning rate to be 0.1 for all datasets

As an example, the following two command lines would run your programs on the large dataset in the handout for 500 epochs. You are given the output of this command and the equivalent command on the small dataset in the handout directories `largeoutput` and `smalloutput`.

```
$ python feature.py \
largedata/train_large.tsv \
largedata/val_large.tsv \
largedata/test_large.tsv \
glove_embeddings.txt \
largeoutput/formatted_train_large.tsv \
largeoutput/formatted_val_large.tsv \
largeoutput/formatted_test_large.tsv

$ python lr.py \
largeoutput/formatted_train_large.tsv \
largeoutput/formatted_val_large.tsv \
largeoutput/formatted_test_large.tsv \
largeoutput/formatted_train_labels.txt \
largeoutput/formatted_test_labels.txt \
largeoutput/formatted_metrics.txt \
500 \
0.1
```

Important Note: You will not be writing out the predictions on validation data, only on train and test data. The validation data is *only* used to give you an estimate of held-out negative log-likelihood at the end of each epoch during training. You are asked to graph the negative log-likelihood vs. epoch of the validation and training data in Programming Empirical Questions section. ^a

^aFor this assignment, we will always specify the number of epochs. However, a more mature implementation would monitor the performance on validation data at the end of each epoch and stop SGD when this validation log-likelihood appears to have converged. You should *not* implement such a convergence check for this assignment.

10.6 Starter Code

To help you start this assignment, we have provided starter code in the handout.

10.7 Gradescope Submission

You should submit your `feature.py` and `lr.py` to Gradescope. *Note:* please do not zip them or use other file names. This will cause problems for the autograder to correctly detect and run your code. Gradescope will also provide **hints for common bugs**; Ctrl-F for HINT if you did not receive a full score.